A Retrieval-Augmented Generation System for SUSTech Campus Knowledge QA

Zheng Renjie [1] **Chen Ximing** [1] **Jiang Hongjun** [1]

Abstract

This project builds a Retrieval-Augmented Generation (RAG) question answering system for Southern University of Science and Technology (SUSTech) campus knowledge. The system collects campus-related web documents, constructs a QA-style knowledge base, indexes it with dense embeddings and FAISS, and answers user questions through a locally deployed Qwen3-32B model. To improve retrieval quality, the system supports BGE reranking and LLM-based query rewriting. We evaluate four settings: direct LLM answering, basic RAG, RAG with reranking, and RAG with both query rewriting and reranking. On a 30-question SUSTech test set, direct LLM answering reaches only 36.67% correctness with a 66.67% hallucination rate. Basic RAG improves correctness to 80.00% and reduces hallucination to 0.00%. Reranking further improves correctness to 86.67%, while the rewrite-plus-rerank setting achieves the best retrieval quality with Recall@5 of 96.67%, MRR@5 of 94.44%, and nDCG@5 of 95.86%. These results show that RAG substantially improves factual accuracy, grounding, and source traceability for university-specific question answering.

1. Introduction

Large language models (LLMs) are strong general-purpose question answering systems, but they are unreliable when the question depends on local, institution-specific, or frequently updated knowledge. Questions about SUSTech housing, course selection, library opening hours, student cards, scholarships, and examination rules require exact information from official campus documents. A direct LLM may answer fluently while relying on generic university knowledge, which can cause hallucinated or outdated answers.

This project develops a practical RAG system for SUSTech campus knowledge question answering. Instead of asking the LLM to answer from parametric memory alone, the system first retrieves relevant evidence from a SUSTech knowledge base and then asks the LLM to answer based on the retrieved context. The project is motivated by the general RAG framework proposed by Lewis et al. (2020), but focuses on a local university knowledge scenario.

Table 1: Knowledge base statistics.

Item	Count
Raw campus documents	52
QA records	1700
Indexed FAISS items	1700
Evaluation questions	30

The project studies three research questions:

1. Does RAG improve answer correctness compared with direct LLM answering?
2. Do reranking and query rewriting improve retrieval quality?
3. Can the final system reduce hallucination and provide more traceable answers?

2. System Overview

The implemented system contains five major components: data collection, knowledge base construction, dense retrieval, optional query rewriting and reranking, and answer generation. Figure 1 shows the full pipeline. The upper part is the online answering path, while the lower part shows the offline knowledge base construction process.

3. Knowledge Base

The knowledge base is built from SUSTech-related web documents and processed into QA-style records. Each item keeps the question, answer, source title, and source URL when available. The final collection contains 52 raw documents and 1700 indexed QA records.

The indexed items are designed to be short and retrieval-friendly. Compared with storing long documents directly, the QA-style format makes the dense retriever more likely to match user questions with relevant evidence. This is especially useful for campus service information, where

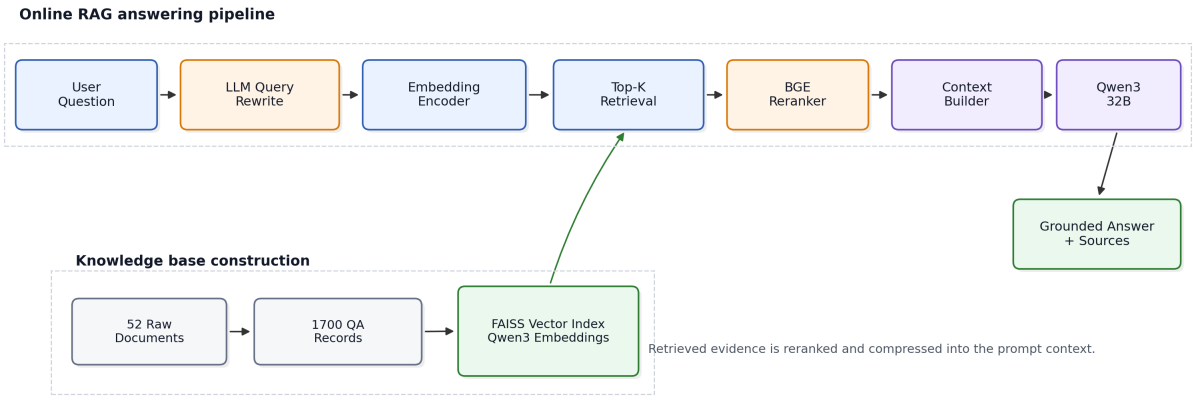

Figure 1: Architecture of the SUSTech campus RAG system. The online pipeline rewrites the user question, retrieves evidence from FAISS, reranks retrieved candidates, builds a compact context, and asks Qwen3-32B to generate a grounded answer with sources.

Table 2: Main system configuration.

Component	Configuration
Generator	Qwen3-32B served by vLLM
Embedding	Qwen3-Embedding-0.6B
Index	FAISS dense vector index (Johnson et al., 2019)
Retriever	TOP_K=30
Reranker	bge-reranker-v2-m3
Context	Top 5 chunks after reranking
Frontend	Gradio web application

users often ask short natural-language questions rather than keyword-style queries.

4. Implementation

The system is deployed on a remote GPU server. The generation model is served through vLLM and accessed through an OpenAI-compatible chat-completion API (Kwon et al., 2023). Embedding and reranking are executed in the RAG service process. A Gradio interface is used for interactive demonstration and qualitative testing.

4.1. Dense Retrieval

For each user question, the system first encodes the query into a dense vector and searches the FAISS index for the top candidate QA records. The retriever returns a relatively large candidate set, which gives the reranker enough evidence to reorder. In this project, the initial retrieval size is set to 30.

4.2. LLM Query Rewriting

To make retrieval more robust and fair, each evaluation question is normalized to explicitly mention SUSTech. This avoids a setting where the direct LLM can answer using generic knowledge about other universities. The query rewriting module then converts the qualified user question into a retrieval-oriented SUSTech query.

For example, a user question about who can live in on-campus faculty apartments is rewritten into a more explicit query about the target residents of SUSTech faculty apartments. The rewritten query is used only for retrieval and reranking. The original qualified question is still used for final answer generation and evaluation.

4.3. Reranking

After initial retrieval, the BGE reranker reorders the retrieved candidates according to their relevance to the query. The final context is constructed from the top five reranked chunks. This step increases the probability that the generator receives the most useful evidence, especially when the initial dense retriever returns partially relevant or duplicated records.

5. Evaluation Design

5.1. Compared Systems

Four systems are compared on the same 30-question test set.

- **Direct**: Qwen3-32B answers directly without retrieval.
- **Basic RAG**: FAISS retrieval followed by Qwen3-32B

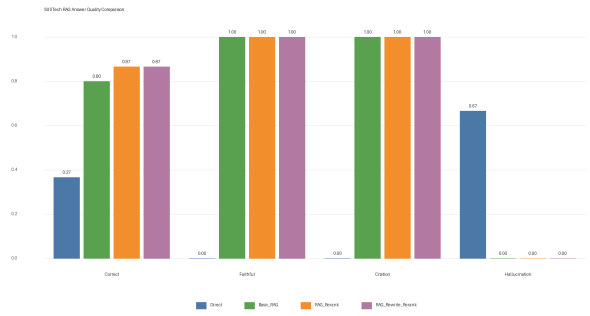

Figure 2: Answer quality comparison across the four systems. RAG improves correctness and removes hallucinations in this test set.

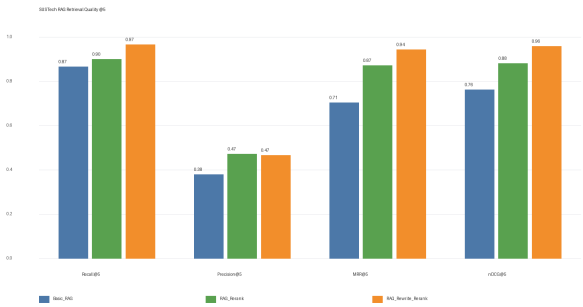

Figure 3: Retrieval quality comparison. Query rewriting improves Recall@5 and ranking metrics.

generation.

- **RAG Rerank**: FAISS retrieval, BGE reranking, and generation.
- **RAG Rewrite Rerank**: LLM query rewriting, FAISS retrieval, BGE reranking, and generation.

5.2. Metrics

The evaluation separates retrieval quality and generation quality. Retrieval is measured by Recall@K, Precision@K, MRR@K, nDCG@K, evidence rank, and context relevance. Generation is measured by answer correctness, relevance, completeness, faithfulness, groundedness, citation accuracy, hallucination rate, and refusal behavior. A score of 2 or higher on a 0–3 scale is treated as passing. End-to-end latency is also recorded.

6. Results

6.1. Answer Quality

Table 3 and Figure 2 show answer quality. Direct LLM answering performs poorly on SUSTech-specific questions, with only 36.67% correctness and a 66.67% hallucination rate. Basic RAG improves correctness to 80.00% and reduces hallucination to 0.00%. Reranking further improves correctness to 86.67%. Query rewriting does not further improve final correctness on this small test set, but it improves answer relevance and completeness.

6.2. Retrieval Quality

Table 4 and Figure 3 show retrieval quality. Reranking significantly improves ranking quality: MRR@5 increases from 70.56% to 87.33%, and nDCG@5 increases from 76.23% to 88.30%. Query rewriting further improves Recall@5 to 96.67%, MRR@5 to 94.44%, and nDCG@5

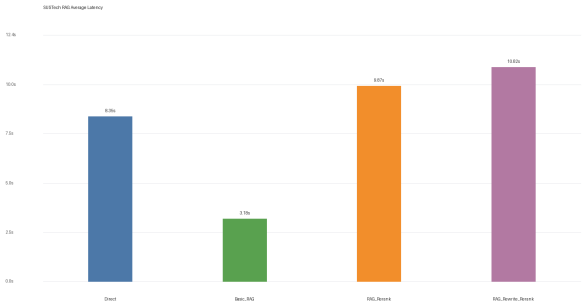

Figure 4: Average end-to-end latency.

to 95.86%. This indicates that rewrite-plus-rerank is the strongest retrieval configuration.

6.3. Latency Trade-off

Figure 4 shows the latency trade-off. Basic RAG is the fastest RAG system because it avoids extra reranking and rewriting. Rewrite-plus-rerank achieves the best retrieval quality but requires an additional LLM call and reranker inference. In practice, Basic RAG is suitable for faster interaction, while rewrite-plus-rerank is preferable when evidence quality is more important.

7. Case Study

7.1. Successful Grounding

For a question about who can live in SUSTech on-campus faculty apartments, the direct LLM gives a plausible but generic answer: it says the apartments are mainly for university faculty and staff. The gold answer is more specific: they are mainly for teaching and research series personnel at assistant professor level or above.

The rewrite-plus-rerank system retrieves the official housing page and answers with the specific target group. This exam-

Table 3: Answer quality comparison. Correctness pass means answer correctness score ≥ 2.

System	Correct. Pass	Avg. Correct.	Faithful Pass	Citation Pass	Hallucination	Latency
Direct	36.67%	1.17	–	–	66.67%	8.35s
Basic RAG	80.00%	2.40	100.00%	100.00%	0.00%	3.18s
RAG Rerank	86.67%	2.60	100.00%	100.00%	0.00%	9.87s
RAG Rewrite Rerank	86.67%	2.60	100.00%	100.00%	0.00%	10.82s

Table 4: Retrieval quality at top 5.

System	R@5	P@5	MRR@5	nDCG@5
Basic RAG	86.67	38.00	70.56	76.23
RAG Rerank	90.00	47.33	87.33	88.30
Rewrite Rerank	96.67	46.67	94.44	95.86

ple shows the key benefit of RAG: the answer is grounded in a campus-specific source instead of generic model memory.

7.2. Failure Analysis

The system still fails on several questions. Representative examples include the 2022-and-later general elective credit requirement, library branch opening hours, and national scholarship policy questions. Some errors are retrieval failures, while others are caused by incomplete or conflicting context. This suggests that knowledge base coverage and evidence annotation should be improved before deploying the system in a real campus setting.

8. Discussion

The experiments support three findings. First, RAG greatly improves reliability for local knowledge: direct answering is weak because campus details are not reliably stored in model parameters. Second, reranking improves evidence ordering and therefore improves downstream correctness. Third, query rewriting improves retrieval recall but adds latency. Better retrieval is necessary but not always sufficient: when retrieved context is incomplete or conflicting, the generator may still refuse or produce an incomplete answer.

9. Limitations

This project has several limitations. The evaluation set has only 30 questions, so the results should be interpreted as project-level evidence rather than a large-scale benchmark. LLM-as-a-judge is scalable but may introduce scoring variance. More unanswerable questions should be added to evaluate refusal behavior. Finally, the knowledge base quality determines the system ceiling: missing, duplicated, or outdated documents can still lead to wrong answers.

10. Conclusion

This project implements and evaluates a complete RAG system for SUSTech campus knowledge question answering. Compared with direct Qwen3-32B answering, RAG improves answer correctness from 36.67% to at least 80.00% and reduces hallucination from 66.67% to 0.00%. Reranking further improves correctness to 86.67%. LLM query rewriting achieves the best retrieval quality, reaching 96.67% Recall@5 and 95.86% nDCG@5. Overall, the system demonstrates that retrieval augmentation is highly effective for institution-specific question answering, especially when factual accuracy, grounding, and source traceability are required.

References

Johnson, J., Douze, M., and Jégou, H. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 2019.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM Symposium on Operating Systems Principles*, 2023.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.